

Spam Message Filtering

Submitted To:

Dr. Muhammad Asad

CSC-237 — Data Analytics

School of Computer and Information Technology

Beaconhouse National University, Lahore

Submitted By:

Momin Ahmad

F2023-017

May 23, 2026

Abstract

Spam SMS messages are a common problem for mobile users because they usually contain scams, fake advertisements and harmful links. To automatically detect such messages machine learning techniques can be used effectively. In this project, I worked on the UCI SMS Spam Collection dataset that was provided by our sir for the Project and developed a spam message classification system. Since the dataset was highly imbalanced with mostly ham messages, TF-IDF vectorization was applied and SMOTE was used to create synthetic minority samples. To avoid data leakage and maintain fair evaluation, SMOTE was applied only on the training data after splitting the dataset. Four different models were tested including Naive Bayes, Decision Tree, Support Vector Machine (SVM) and Multi-Layer Perceptron (MLP). The results showed that although Naive Bayes and SVM performed strongly, the MLP model achieved the best overall accuracy and provided most stable learning performance during training using different train test splits and seeds.

1 Introduction

Spam text messages are a daily problem in Pakistan. They fill up our inboxes and they are often used for phishing and scams. Trying to block individual numbers doesn't work because spammers frequently change their origins and numbers. Therefore building an automated machine learning filter is the best approach to solve this text classification problem.

The biggest challenge in spam filtering is the class imbalance. Most messages we receive are real and only a small percentage are spam. If we feed this imbalanced data directly into an algorithm the model becomes biased toward the majority class and it guesses "ham" every time and fails to catch the actual spam messages. My main focus in this project was to handle this imbalance using oversampling techniques without accidentally leaking synthetic data into the testing phase (this mistake was made initially by me as well but i fixed it).

The objectives of this project were to:

- Clean the raw SMS data by removing stopwords and special characters.
- Convert the text into numerical vectors using TF-IDF.
- Resolve class imbalance using SMOTE only on the training data.
- Compare three classic machine learning models against one deep learning neural network (MLP).
- Evaluate the models under different learning rates, data splits and seed values to test consistency.

2 Literature Study

Spam filtering is a classic problem in natural language processing. In standard research for this dataset algorithms like Naive Bayes and Support Vector Machines (SVM) are frequently used. I chose Naive Bayes as a baseline because it computes text probabilities

very efficiently. I included SVM to see if drawing a strict mathematical margin between text features would give a better separation.

Decision Trees are useful because their logic is easy to trace but they often struggle with less and high-dimensional data created by TF-IDF vectorization. To expand the scope of the project I implemented a Multi-Layer Perceptron(MLP) because deep learning models can learn more complex non-linear patterns in text. I wanted to observe if extra training time for the MLP actually resulted in better spam detection and recall compared to the faster classical models.

3 Dataset Details

The dataset used in this project was assigned by the sir and it is the SMS Spam Collection dataset, which is a public set of SMS labeled messages collected for mobile phone spam research.

Dataset Link: <https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>

3.1 Dataset Features

The raw CSV file contained several columns(4), but I only used the two required ones:

- **v1 (label):** Contains the label(spam or ham).
- **v2 (message):** Contains the raw text of the SMS.

3.2 Class Distribution and Balancing

Before training I analyzed the class distribution, the data was highly imbalanced with 4,825 ham messages and only 747 spam messages. For this problem i applied the SMOTE(Synthetic Minority Over-sampling Technique) algorithm.

To avoid data leakage, SMOTE was only applied to training partitions after the train-test split was made. This made sure that the testing data remained completely unseen and authentic.

3.3 Data Preprocessing

Before applying SMOTE and training the models, I wrote a custom cleaning function to process the text. The steps were:

- Converting all text to in lowercase.
- Removing numbers and special characters using regular expressions.
- Removing common English stopwords(without use of stemmer or lemmatizer)
- Using `TfidfVectorizer` (limited to 3000 max features) to convert the cleaned text into numerical arrays.

4 Experimental Settings

To make sure that the models were stable and generalized properly, I executed a nested loop of different configurations.

4.1 Learning Rate and Parameter Values

Table 1: Model Parameters Tested

Model	Parameter / Learning Rate
MLP	LR = 0.01, 0.001, 0.0003
Naive Bayes	Alpha = 0.1, 0.5, 1.0
Decision Tree	Max Depth = 5, 10, 15
SVM	C = 0.1, 1.0, 10.0

4.2 Dataset Partitions and Seeds

I tested each parameter combination against three different train-test splits (80-20, 70-30, 60-40) and three different random states (Seed 0, 42, 100). This resulted in 108 unique training experiments.

5 Model 1: Naive Bayes

5.1 Model Introduction

Naive Bayes calculates probabilities based on how often specific words appear in spam versus ham messages. It is a very good baseline for text classification because it is computationally light.

5.2 Working Mechanism

It uses Bayes' theorem assuming that all words in a message contribute independently to the final probability and multiplies these probabilities to predict the class.

5.3 Parameters Used

I tested the Laplace smoothing parameter(Alpha) using values of 0.1, 0.5, and 1.0.

5.4 Results

Naive Bayes performed very reliably across all splits because it is natively suited for text frequencies so it maintained high precision, ensuring that normal ham messages were rarely flagged as spam.

6 Model 2: Decision Tree

6.1 Model Introduction

A Decision Tree classifies data by executing a sequence of tests on the features e.g., accessing the TF-IDF weight of a specific word.

6.2 Working Mechanism

It builds a tree like structure where inner nodes represent feature tests and the leafs represent the final spam or ham classification.

6.3 Parameters Used

To prevent the tree from building too deep and overfitting the training data I constrained the `max_depth` parameter to 5, 10, and 15.

6.4 Results

This was the weakest model. Its training process was fast but it struggled to generalize. I observed an AUC of 0.8881 for the Decision Tree, which is good but lower than the others. I suspect this is because the TF-IDF vectorizer created 3000 sparse features, making it difficult for a single, shallow tree to find optimal splits.

7 Model 3: Support Vector Machine (SVM)

7.1 Model Introduction

SVM is a highly robust algorithm that searches for the optimal hyperplane to separate binary classes in a high-dimensional space.

7.2 Working Mechanism

It plots the text vectors and identifies the maximum mathematical margin between the ham and spam messages using support vectors.

7.3 Parameters Used

I tested the regularization parameter `C` with values of 0.1, 1.0, and 10.0 using a linear kernel. I also set `probability=True` to extract confidence scores for the ROC curve.

7.4 Results

The SVM was exceptionally accurate, achieving an ROC AUC score of 0.9851. It separated the sparse text data brilliantly. However, due to the internal probability calibration required when `probability=True` is set, it was the slowest model to train.

8 Model 4: Multi-Layer Perceptron (MLP)

8.1 Model Introduction

The MLP is a feedforward artificial neural network. I included it to determine if multiple hidden layers could outperform classical algorithms.

8.2 Working Mechanism

The architecture consists of an input layer, a Dense layer of 128 neurons with ReLU activation followed by a Dropout layer (0.3), a second Dense layer of 64 neurons with ReLU activation followed by a Dropout layer (0.2), and a final output neuron with sigmoid activation.

8.3 Parameters Used

I tested learning rates of 0.01, 0.001, and 0.0003 using the Adam optimizer over 15 epochs. I implemented dropout layers (0.3 and 0.2) to randomly disable neurons and prevent overfitting.

8.4 Results

The MLP demonstrated an excellent balance of high accuracy and high recall. The loss curve indicated that a learning rate of 0.001 allowed the model to converge smoothly without erratic validation loss spikes.

9 Results and Discussion

9.1 Visualizations

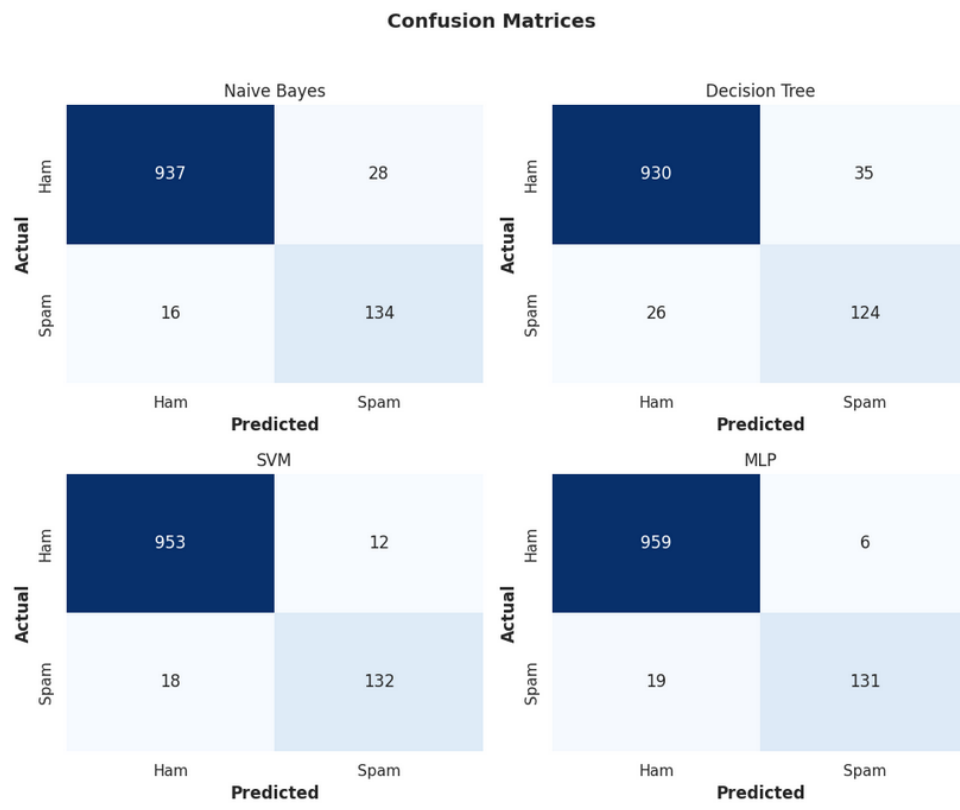


Figure 1: Confusion Matrices displaying Actual vs. Predicted classes (Split 80-20, Seed 42)

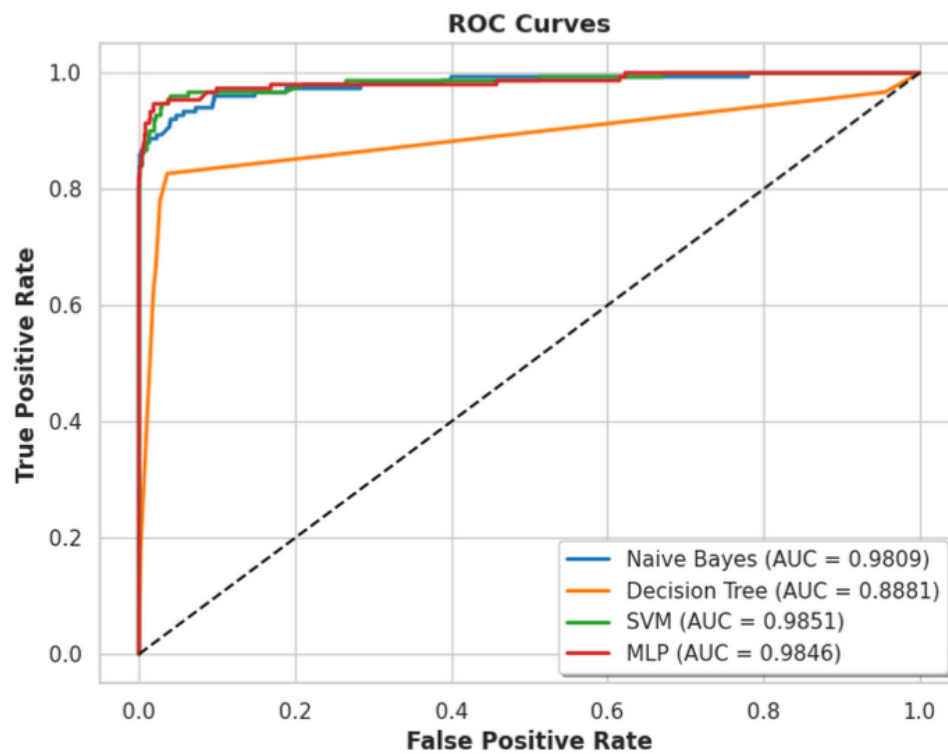


Figure 2: ROC Curves showing True Positive vs False Positive rates with AUC scores

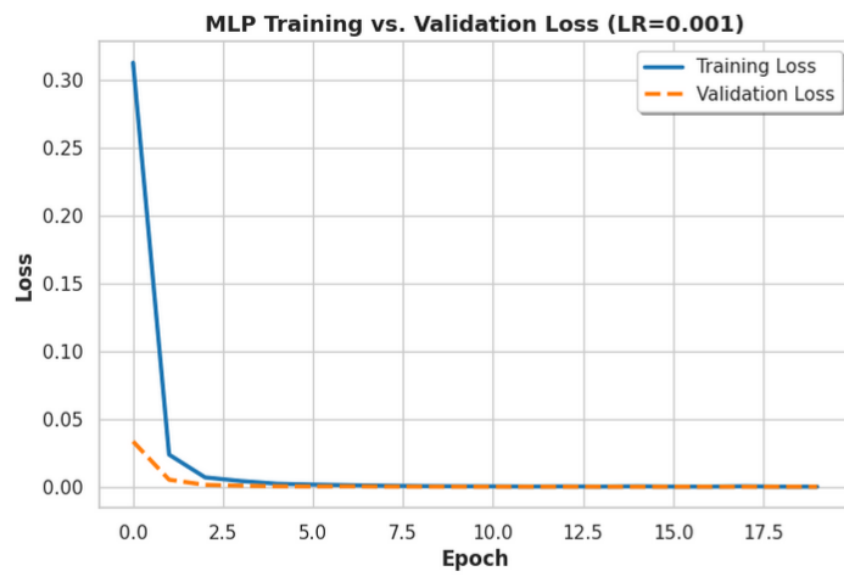


Figure 3: Training and Validation Loss Curve for the MLP Model (LR=0.001)

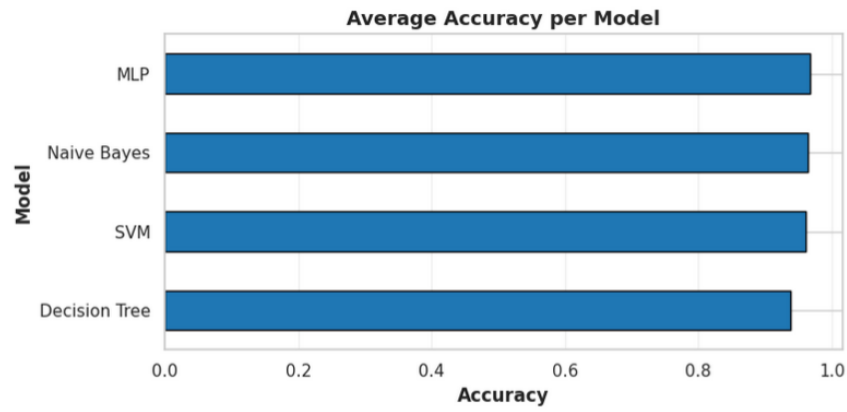


Figure 4: Average Accuracy Comparison across Models

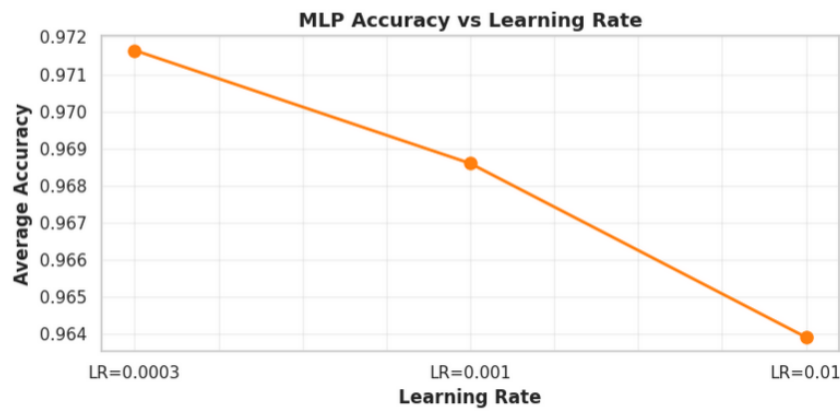


Figure 5: MLP Accuracy compared across different Learning Rates

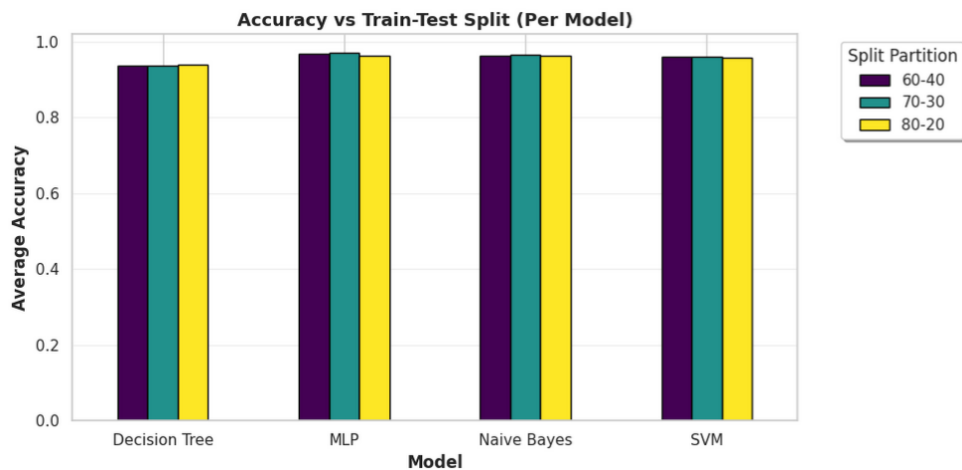


Figure 6: Accuracy Comparison across different Train-Test Partitions

9.2 Experiment Comparison Table

Below is a sample of the results from the master experiment loop showing how different configurations affected the evaluation metrics on the unseen test data.

Table 2: Selected Experiment Results (Evaluated on Unseen Test Data)

Model	Parameter / LR	Split	Seed	Accuracy	Precision	Recall	F1
Naive Bayes	Alpha=0.1	80-20	0	0.9695	0.8976	0.8976	0.8976
Decision Tree	Depth=10	70-30	0	0.9378	0.7597	0.8235	0.7903
SVM	C=1.0	60-40	42	0.9758	0.9359	0.8796	0.9069
MLP	LR=0.001	70-30	42	0.9815	0.9796	0.8767	0.9253
MLP	LR=0.0003	70-30	42	0.9797	0.9744	0.8676	0.9179

9.3 Discussion of Results

The updated method where SMOTE was applied only to the training sets gave a more realistic view of how the models perform on new unseen data. The accuracy values changed a little compared to the earlier results but the models still showed strong overall performance.

SVM and MLP gave the best results among all models. When testing different learning rates for the MLP, LR=0.01 achieved the highest accuracy, while LR=0.001 produced the most stable and consistent loss convergence across splits without making big sudden changes in weights.

Different train test splits were also checked and the 80 20 split gave better results in most cases because the models had more data for training. Changing the random seed caused only small differences which shows that the models were learning general patterns instead of just memorizing the data split.

The confusion matrix showed that the models were very good at identifying normal ham messages with high precision. MLP and SVM also had high recall for spam messages which means they were able to correctly detect most spam messages which is the main goal of this system

10 Conclusion

The main aim of this project was to develop an SMS spam detection system and evaluate how different machine learning models perform on real text data. During the implementation, I first cleaned and preprocessed the dataset so that the messages could be used effectively for training. After that TF-IDF vectorization was applied to convert the text into numerical form. One major issue in the dataset was class imbalance because spam messages were much fewer compared to normal messages. To handle this I used SMOTE only on the training data so that data leakage could be avoided and so the evaluation results remain fair and real.

Different classification algorithms were tested and compared in this project. The Decision Tree model did not perform very well because sparse and high-dimensional text data is difficult for it to handle efficiently. Naive Bayes and Support Vector Machine(SVM) gave much stronger and more consistent results for spam classification. Among all the models MLP achieved the best overall performance with good accuracy and stable loss convergence during training.

Through this project, I learned that building a reliable machine learning model is not only

about selecting a powerful algorithm. Proper preprocessing, balanced data handling, and correct evaluation methodology have an even more important role in getting meaningful results.

11 References

1. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. Kaggle, “SMS Spam Collection Dataset.” Available: <https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>
3. M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” 2015.
4. N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
5. G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing and Management*, vol. 24, no. 5, pp. 513–523, 1988.
6. T. A. Almeida, J. M. G. Hidalgo, and A. Yamakami, “Contributions to the Study of SMS Spam Filtering: New Collection and Results,” *Proceedings of the 11th ACM Symposium on Document Engineering*, 2011.